VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF SCIENCE — VNUHCM
Faculty of Information Technology

PROJECT REPORT — R-TREE SPATIAL INDEXING

(Final Version)

Subject: CS163 - Data Structures and Algorithms

Group:	51
Class:	25A02
Submission Date:	April 18, 2026

Ho Chi Minh City, 2026

Contents

1 Fundamentals of the R-Tree **2**
1.1 Introduction and Historical Context . . . 2
1.2 Structural Characteristics . . . 2
1.2.1 Balance and Symmetry . . . 2
1.2.2 Node Capacity and Occupancy Rules . . . 2
1.2.3 Minimum Bounding Rectangles (MBR) . . . 2
1.3 Optimization Principles in Construction . . . 2
1.3.1 Perimeter vs. Area Minimization . . . 3
1.3.2 The Filter-Refinement Framework . . . 3

2 Algorithms **4**
2.1 2D Orthogonal Range Reporting (Window Query) . . . 4
2.1.1 Query Logic and Spatial Pruning . . . 4
2.1.2 Query Algorithm . . . 4
2.2 Node Insertion . . . 5
2.2.1 Choose Subtree . . . 5
2.2.2 Overflow and Node Splitting . . . 5
2.2.3 Parental Updates and Propagation . . . 6
2.2.4 Splitting Node . . . 6
2.3 Node Deletion . . . 7
2.3.1 Remove the index record E from R-Tree . . . 7
2.3.2 Find Leaf . . . 7
2.3.3 Condense Tree . . . 7

3 Complexity Comparison **9**
3.1 1. Execution Time / Speed Evaluation . . . 9
3.2 2. Memory Usage Evaluation . . . 10

4 Implementation and Libraries **11**
4.1 Source Codes . . . 11
4.2 Production-Ready Implementation: Boost.Geometry . . . 11
4.2.1 Key Advantages and Features . . . 11
4.2.2 Built-in Functions (API) . . . 11
4.2.3 Installation and Integration . . . 12
4.2.4 Basic Setup and Data Type Definition . . . 12
4.2.5 Tree Initialization and Data Insertion . . . 13
4.2.6 Executing Spatial Queries . . . 13
4.3 Datasets . . . 14

5 Programming Assignments **15**
5.1 Exercise 1 . . . 15
5.2 Exercise 2: Region Query on a Set of Rectangles . . . 16
5.3 Exercise 3: K-Nearest Neighbors (KNN) . . . 16
5.4 Exercise 4: Point Enclosure Query (Application in Mouse Click Detection on UI) . . . 17

Chapter 1

Fundamentals of the R-Tree

1.1 Introduction and Historical Context

The R-tree is a hierarchical, multidimensional indexing structure specifically designed to manage spatial data such as points, line segments, and polygons. Introduced by Antonin Guttman of the University of California, Berkeley, in 1984, the R-tree serves as a multidimensional extension of the B-tree (specifically the B+ tree variant). While traditional indexing structures are optimized for one-dimensional data, the R-tree excels in processing window queries or orthogonal range reporting, making it a cornerstone of modern Geographic Information Systems (GIS) and spatial databases.

1.2 Structural Characteristics

1.2.1 Balance and Symmetry

Mirroring the properties of B-trees, R-trees are perfectly height-balanced, ensuring that all leaf nodes reside at the same level. This balance is maintained dynamically through specific occupancy rules governed by a parameter B, representing the maximum node capacity.

1.2.2 Node Capacity and Occupancy Rules

For an R-tree of order B (where $B \geq 3$), the following constraints apply to maintain tree density and search efficiency:

- **Leaf Nodes:** Each leaf contains between $0.4B$ and B data objects. The only exception is the root, which may contain between 1 and B objects.
- **Internal Nodes:** Each internal node possesses between $0.4B$ and B children. If the root is an internal node, it must have at least 2 children.

In disk-resident implementations, B is typically chosen so that a node's size corresponds to a single disk block, minimizing I/O overhead.

1.2.3 Minimum Bounding Rectangles (MBR)

The fundamental unit of spatial organization in an R-tree is the Minimum Bounding Rectangle (MBR). For any node u, let S_u be the set of all spatial objects within its subtree. An internal node u with children $v_1, v_2, \ldots, v_f$ (where $f \leq B$) stores an entry for each child consisting of:

1. A pointer to the child node v_i
2. $MBR(v_i)$, the smallest axis-parallel rectangle that tightly encloses all objects in the subtree S_{v_i}

1.3 Optimization Principles in Construction

The R-tree definition does not strictly dictate how objects should be grouped into nodes. However, the quality of grouping significantly impacts performance.

1.3.1 Perimeter vs. Area Minimization

A core heuristic in R-tree construction is the minimization of the **perimeter sum** of all MBRs rather than their total area.

- **Query Efficiency:** Rectangles with smaller perimeters tend to be more square-like, reducing the probability of overlap with other MBRs or the query window q.
- **Geometric Logic:** While minimizing perimeter often minimizes area, the inverse is not always true. A rectangle can have a very small area but a large perimeter (e.g., a long, thin "needle" shape. Such shapes are inefficient for spatial indexing as they are more likely to intersect search regions across large distances.

1.3.2 The Filter-Refinement Framework

In complex applications like road segment retrieval, the R-tree is often used within a Filter-Refinement framework:

1. **Filter Step:** The R-tree quickly identifies a *Candidate Set* of MBRs that intersect the query window. This step is ($\mathcal{O}(\log N + k)$), but may include False Hits.
2. **Refinement Step:** A secondary geometric check is performed on the candidate set to determine if the actual spatial object (e.g., a diagonal line) truly intersects the query window.

Chapter 2

Algorithms

This chapter analyzes the core algorithms that maintain the R-tree's structural integrity and facilitate efficient multidimensional data retrieval. These mechanisms include spatial searching, point insertion, deletion and node splitting.

2.1 2D Orthogonal Range Reporting (Window Query)

The primary operation supported by the R-tree is 2D Orthogonal Range Reporting, or a window query. This is essential for applications requiring data retrieval within specific geographic or attribute-defined boundaries, such as map displays.

Problem Definition Let S be a set of n axis-parallel rectangles in a 2D Euclidean space ($S \subset \mathbb{R}^2$). Given an axis-parallel query rectangle q, the objective is to return the subset of rectangles $r \in S$ that intersect q:

$$Result = \{r \in S \mid r \cap q \neq \emptyset\}$$

2.1.1 Query Logic and Spatial Pruning

The R-tree executes this query via a recursive search starting at the root. Efficiency is achieved through spatial pruning, where the algorithm compares q against the MBRs of a node's children.

- **Pruning Phase:** If a child's MBR does not intersect the query window q, the entire subtree rooted at that child is excluded from the search.
- **Refinement Phase:** If an intersection exists, the algorithm recursively descends into the child node. At the leaf level, the actual data rectangles are verified for inclusion within q.

2.1.2 Query Algorithm

The search process is recursive and leverages the MBR hierarchy to prune subtrees that do not intersect the search region q

Algorithm 1 Range Query Algorithm

```
procedure RangeQuery(u, q)
    if u is a leaf node then
        report all points p in u such that p is covered by q
    else
        for each child v of u do
            if MBR(v) intersects q then
                RangeQuery(v, q)
            end if
        end for
    end if
end procedure
```

For a dataset containing N objects and a tree with a maximum branching factor (order) B, the time complexity of an orthogonal range query is generally expressed as:

$$\mathcal{O}(\log_B N + k)$$

where:

- $\log_B N$: the height of the tree, determining the number of nodes visited from the root to the leaves.
- k: number of objects found within the query results (output sensitivity) .

The following sections detail tree adaptation through insertion algorithms and the heuristics used during node overflows to maintain query performance.

2.2 Node Insertion

Inserting a new spatial object into an R-tree involves a top-down traversal to locate the most suitable leaf node, followed by a potential bottom-up propagation of structural changes. The process is governed by two primary goals: maintaining the tree's height balance and minimizing the expansion of MBRs to preserve query efficiency.

Algorithm 2 R-Tree Insertion

```
procedure INSERT(u, p)
    if u is a leaf node then
        Add p to u
        if u has B + 1 points then                    ▷ Overflow condition
            HANDLEOVERFLOW(u)
        end if
    else
        v ← CHOOSESUBTREE(u, p)
        INSERT(v, p)
        Update MBR(v) in u to enclose p
    end if
end procedure
```

2.2.1 Choose Subtree

Before the actual insertion, the algorithm must decide which leaf node should accommodate the new object. Starting from the root, the algorithm recursively selects the child node whose MBR requires the minimum increase in perimeter to enclose the new object. In the event of a tie, the child with the smallest existing MBR is favored.

Algorithm 3 ChooseSubtree Algorithm

```
procedure CHOOSESUBTREE(u, p)
    if u is a leaf node then
        return u
    end if
    Select the child v of u whose MBR(v) requires the minimum increase in perimeter to enclose p.
    if multiple children result in the same minimum increase then
        Tie-break: Select the child with the smallest MBR area.
    end if
    return CHOOSESUBTREE(v, p)
end procedure
```

2.2.2 Overflow and Node Splitting

If the selected leaf node is already at its maximum capacity B, an **overflow** occurs. To resolve this, the node is split into two, and the $B + 1$ entries are redistributed using a heuristic split algorithm (e.g., Linear, Quadratic, or Greene's split) aimed at minimizing the resulting MBR perimeters.

Algorithm 4 Handle Overflow

```
procedure HandleOverflow(u)
    Split u into nodes u and u'
    if u is the root then
        Create a new root with u and u' as children
    else
        w ← parent of u
        Update MBR(u) in w
        Add u' as a child of w
        if w overflows then
            HandleOverflow(w)
        end if
    end if
end procedure
```

2.2.3 Parental Updates and Propagation

Any modification to a leaf requires updating the corresponding MBR in the parent node to ensure it remains a minimum bounding container. If a split occurred, the parent must also accommodate a new entry for the second node. If the parent is full, the overflow handling propagates upward, potentially reaching the root. If the root splits, a new one is created and the tree's height increases by one.

2.2.4 Splitting Node

- **Splitting a leaf**

  Essentially we are dealing with the following problem:

  > Let S be a set of $B+1$ points. Divide S into two disjoint sets S_1 and S_2 to minimize the perimeter sum of $MBR(S_1)$ and $MBR(S_2)$, subject to the condition that $|S_1| \geq 0.4B$ and $|S_2| \geq 0.4B$.

Algorithm 5 split u

```
1:  procedure SplitNode(u)
2:      m = the number of the points in u
3:      sort the points of u on x-dimension
4:      for i = [0.4B] to m − [0.4B] do
5:          S1 is the set of the first i points in the list
6:          S2 is the set of the other m − i points in the list
7:          calculate the perimeter sum of MBR(S1) and MBR(S2); record it if this is the best split so far
8:      end for
9:      Repeat Lines 2–6 with respect to y-dimension
10:     return the best split found
11: end procedure
```

- **Splitting the internal node**

Algorithm 6 split u

```
1: procedure SplitInternalNode(u)
2:     m = the number of points in u
3:     sort the rectangles in u by their left boundaries on the x-dimension
4:     for i = [0.4B] to m − [0.4B] do
5:         S1 is the set of the first i rectangles in the list
6:         S2 is the set of the other m − i rectangles in the list
7:         Calculate the perimeter sum of MBR(S1) and MBR(S2); record it if this is the best split so far
8:     end for
9:     Repeat Lines 2–6 with respect to the right boundaries on the x-dimension
10:    Repeat Lines 2–7 w.r.t. the y-dimension
11:    return the best split found
12: end procedure
```

2.3 Node Deletion

2.3.1 Remove the index record E from R-Tree

Delete(T,E)

Algorithm 7 Delete

```
procedure Delete(T, E)
    L = FindLeaf(T, E)
    if L ≠ NULL then
        Remove E from L;
        CondenseTree(L);
    end if
    if root node has 1 child then
        make the child the new root
    end if
end procedure
```

2.3.2 Find Leaf

FindLeaf() Given an R-tree whose root node is T, find the leaf node containing the index entry E.

Algorithm 8 FindLeaf

```
procedure FindLeaf(T, E)
    Search subtrees
    if T is not a leaf then
        check each entry F in T to determine if F_i overlaps E.
        For each such entry invoke FindLeaf on the tree whose root is pointed to by F_p until E is found or
    all entries have been checked.
    end if
    Search leaf node for record
    if T is a leaf then
        check each entry to see if it matches E.
        if E is found then
            return T
        end if
    end if
end procedure
```

2.3.3 Condense Tree

CondenseTree() Given a leaf node L from which an entry has been deleted, eliminate the node if it has too few entries and relocate its entries. Propagate node elimination upward as necessary. Adjust all covering rectangles

on the path to the root, making them smaller if possible.

Algorithm 9 CondenseTree

```
procedure CondenseTree(L)
    Set N = L. Set Q, the set of eliminated nodes, to be empty.
    if N is the root then
        go to Step 6.
    else
        let P be the parent of N and let EN be N's entry in P.
    end if
    if N has fewer than m entries then
        delete EN from P and add N to set Q.
    end if
    if N has not been eliminated then
        adjust ENI to tightly contain all entries in N.
    end if
    Set N = P and repeat from CT2.
    Re-insert all entries of nodes in set Q. Entries from eliminated leaf nodes are re-inserted in tree leaves
    as described in Algorithm Insert, but entries from higher-level nodes must be placed higher in the tree, so
    that leaves of their dependent subtrees will be on the same level as leaves of the main tree.
end procedure
```

Chapter 3

Complexity Comparison

This section presents a comparison of the running time and memory usage between the naive solution (brute-force) and the R-Tree algorithm on large-scale datasets.

3.1 1. Execution Time / Speed Evaluation

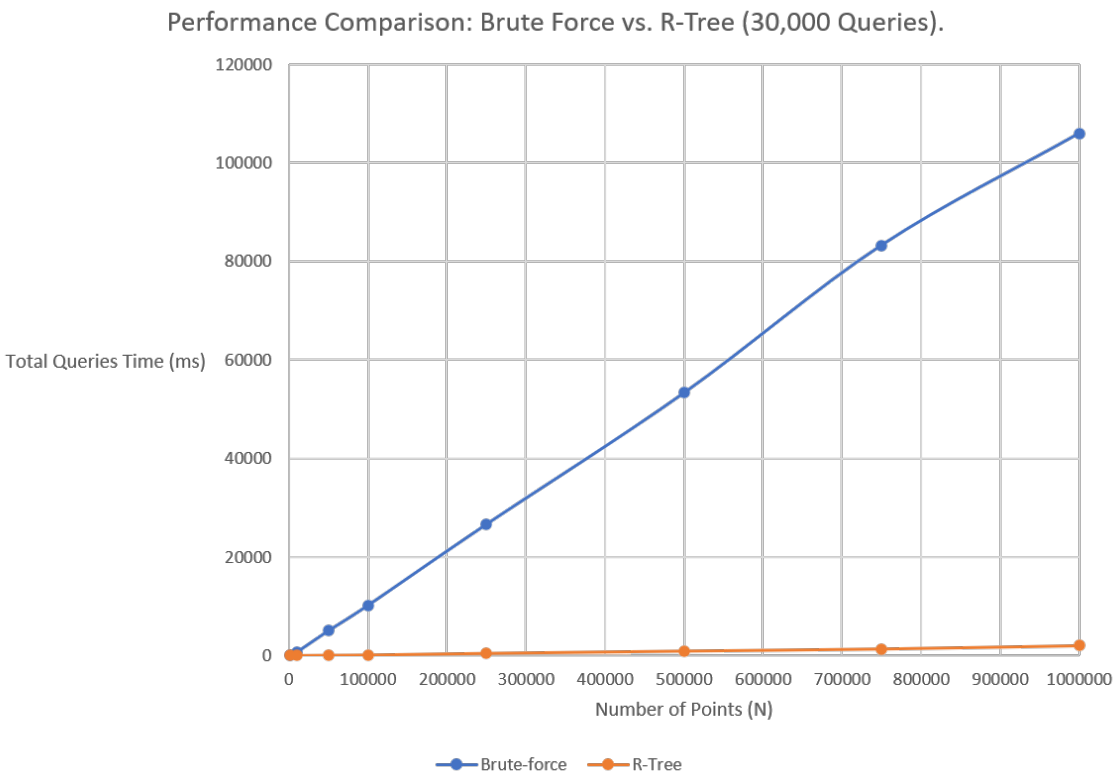

Figure 3.1: Execution time comparison between Brute-force and R-Tree

The chart clearly illustrates the difference in time complexity between the two algorithms when executing 30,000 queries. The Brute-force method exhibits linear growth $\mathcal{O}(N)$, leading to a sharp increase in running time, taking nearly 106 seconds (105,970 ms) to complete queries on a dataset of 1 million points. Conversely, the R-Tree structure proves its optimization through the spatial pruning mechanism with an asymptotic complexity of $\mathcal{O}(\log N)$. The query time of the R-Tree grows extremely slowly, taking only about 2 seconds (2026.34 ms) for the same data scale. At the $N = 1,000,000$ mark, the R-Tree achieves a speedup approximately 52 times faster than Brute-force, confirming the superior performance of indexing when handling a massive number of queries.

3.2 2. Memory Usage Evaluation

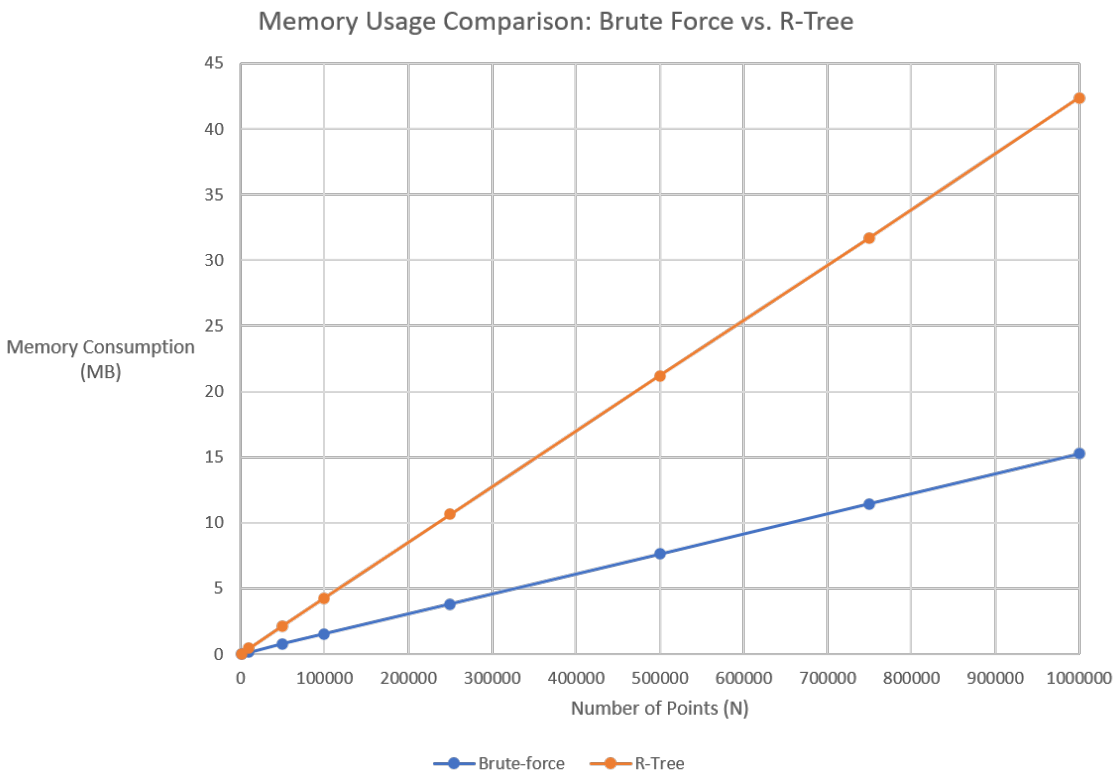

Figure 3.2: Memory usage comparison between Brute-force and R-Tree

The chart demonstrates the mandatory trade-off in storage space to exchange for query speed. Both methods exhibit linear growth in RAM consumption according to the number of points. The Brute-force method perfectly optimizes space by only storing coordinates in an array (costing 15.26 MB for 1 million points). Meanwhile, the R-Tree consumes about 42.38 MB, which is 2.7 to 2.8 times higher than the linear array. This memory overhead arises because the system must allocate extra memory to maintain the hierarchical structure of the tree, including storing the coordinates of the Minimum Bounding Rectangles (MBRs) at internal nodes and the pointers linking parent and child nodes. Despite consuming more space, this is a completely worthwhile and feasible trade-off on modern computer systems.

Chapter 4

Implementation and Libraries

4.1 Source Codes

The entire source code, including the header of the R-Tree data structure, the `main.cpp` file for performance evaluation (clearly displaying critical parameters and comparing directly with the brute-force algorithm), along with the Python source code, have been fully uploaded and organized into a complete repository.

Details on how to run (instructions) are clearly presented in the project's README file.

Access the source code on GitHub: `https://github.com/vbthg/R-tree`

4.2 Production-Ready Implementation: Boost.Geometry

While implementing an R-Tree from scratch is essential for understanding its underlying spatial pruning mechanics, industry applications typically rely on heavily optimized, battle-tested libraries to handle massive geographic datasets. In the C++ ecosystem, the industry standard for spatial indexing is the `boost::geometry::index::rtree` module from the widely acclaimed Boost C++ Libraries.

4.2.1 Key Advantages and Features

The Boost.Geometry R-Tree provides several distinct advantages that make it suitable for production environments:

- **Multidimensional Data Support:** Unlike basic implementations restricted to 2D planes, Boost natively supports indexing in 2D, 3D, and even higher-dimensional spaces.
- **Algorithmic Flexibility:** It provides built-in support for various tree balancing algorithms out-of-the-box, allowing developers to choose between Linear, Quadratic, and the highly optimized R*-Tree (R-Star Tree) depending on the use case.
- **Header-only Integration:** It requires no complex linking processes or the compilation of separate `.dll` or `.lib` binaries. Developers simply include the headers, making it highly portable.
- **Generic Programming:** Built heavily on C++ templates, it can index any custom user-defined data type as long as the spatial properties are properly mapped.
- **Thread Safety:** Concurrent read queries (e.g., multiple users searching the map simultaneously) are completely thread-safe.

4.2.2 Built-in Functions (API)

The library encapsulates complex tree operations into a clean, easy-to-use API. The most commonly used built-in functions include:

- `insert(Value const &)`: Inserts a new spatial object into the tree, automatically handling node splitting if an overflow occurs.
- `remove(Value const &)`: Removes a specific object and handles potential node underflows and tree condensation.

- `query(Predicate, OutIter)`: The core search function. It accepts powerful spatial predicates to filter data without writing manual loops:
  - `bgi::intersects(Geometry)`: Finds all objects overlapping the query geometry (used for Window Queries).
  - `bgi::within(Geometry)`: Finds all objects completely inside the query geometry.
  - `bgi::nearest(Point, k)`: Finds the k nearest neighbors (KNN) to a specific point.
- `size()`: Returns the total number of elements currently stored in the R-Tree in $\mathcal{O}(1)$ time.
- `bounds()`: Returns the Minimum Bounding Rectangle (MBR) of the entire tree (i.e., the MBR of the root node).
- `clear()`: Completely removes all elements and frees the allocated memory.

4.2.3 Installation and Integration

Since the Boost.Geometry R-Tree is a **header-only** library, it does not require a pre-compiled binary. Developers only need to ensure the Boost headers are available in the compiler's search path.

1. **Manual Installation:**
   - Download the Boost source from `https://www.boost.org/`.
   - Extract the archive and add the `/boost` subdirectory to your project's `Include` path in your IDE (e.g., Visual Studio, CLion, or VS Code).
2. **Package Managers (Recommended):**
   - **Windows (vcpkg):** Run `vcpkg install boost-geometry` in the terminal.
   - **Linux (Ubuntu/Debian):** Run `sudo apt-get install libboost-all-dev`.
   - **macOS (Homebrew):** Run `brew install boost`.
3. **Linking:** No extra linking (`-l`) flags are required during compilation. Simply use `#include <boost/geometry.hpp>` in your source files.

4.2.4 Basic Setup and Data Type Definition

To utilize the R-Tree, we must first include the necessary headers and define our spatial geometries (Points and Boxes), along with the payload data we intend to store inside the tree.

```
#include <boost/geometry.hpp>
#include <boost/geometry/index/rtree.hpp>
#include <vector>
#include <iostream>

namespace bg = boost::geometry;
namespace bgi = boost::geometry::index;

// Define a 2D point using Cartesian coordinates
typedef bg::model::point<double, 2, bg::cs::cartesian> Point;
// Define a 2D box (MBR) using the Point type
typedef bg::model::box<Point> Box;
// Define the value type stored in the tree (e.g., an MBR and a Segment ID)
typedef std::pair<Box, int> Value;
```

4.2.5 Tree Initialization and Data Insertion

Boost allows developers to specify the splitting heuristic at compile time. In the following example, we initialize an R-Tree using the Quadratic split algorithm with a maximum node capacity of 16 entries.

```
int main()
{
    // Initialize the R-Tree with a Quadratic splitting algorithm
    bgi::rtree< Value, bgi::quadratic<16> > rtree;

    // Create sample bounding boxes representing road segments
    Box box1(Point(0, 0), Point(2, 2));
    Box box2(Point(5, 5), Point(7, 7));
    Box box3(Point(1, 1), Point(3, 3));

    // Insert data into the R-Tree
    rtree.insert(std::make_pair(box1, 101));
    rtree.insert(std::make_pair(box2, 102));
    rtree.insert(std::make_pair(box3, 103));
```

4.2.6 Executing Spatial Queries

The true power of the Boost R-Tree lies in its expressive and highly optimized querying capabilities, seamlessly supporting both orthogonal range reporting and proximity searches.

1. Window Query (Intersection)

This query is identical to the Filter Step in GIS map rendering, retrieving all segments that overlap with a defined viewing region.

```
    // Define the query window (e.g., the user's screen)
    Box screen_window(Point(1, 1), Point(4, 4));
    std::vector<Value> window_results;

    // Query: Find all items intersecting the screen_window
    rtree.query(bgi::intersects(screen_window), std::back_inserter(window_results));

    // Process the candidate set
    for(auto const& item : window_results)
    {
        if(item.second > 0)
        {
            std::cout << "Candidate Segment ID: " << item.second << std::endl;
        }
    }
```

2. K-Nearest Neighbors (KNN) Query

This query finds a specified number of objects closest to a given geographical point, which is crucial for location-based services like finding nearby drivers.

```
    Point user_location(0, 0);
    std::vector<Value> knn_results;

    // Query: Find the 2 nearest items to the user_location
    rtree.query(bgi::nearest(user_location, 2), std::back_inserter(knn_results));

    for(auto const& item : knn_results)
    {
        std::cout << "Nearby Segment ID: " << item.second << std::endl;
    }

    return 0;
}
```

4.3 Datasets

The large-scale datasets used to run the benchmark have been fully provided in the GitHub repository at the same link above.

Chapter 5

Programming Assignments

5.1 Exercise 1

In a Geographic Information System (GIS) that manages road segments, we are given a list of thousands of line segments (roads) and a rectangular query window representing the user's display screen.
The task is to find all road segments that need to be drawn on the screen.

Solution:

Approach 1: Brute-force

- **Method:** Iterate through all n line segments and check whether each segment intersects with the query rectangle.
- **Complexity:** $\mathcal{O}(n)$ per query, where n is the total number of line segments.
- **Evaluation:** In practice, the system must answer numerous consecutive queries (as the user pans, zooms in/out of the map) $\rightarrow$ This approach is too slow and unfeasible.

Approach 2: Applying the Filter and Refinement Framework with R-Tree

Instead of checking everything, we filter out line segments located far from the query rectangle and only detail-check the "potential" segments. This process consists of 2 standard steps in commercial systems:

Step 1: Filter Step

- **Create MBR:** Represent each line segment with a Minimum Bounding Rectangle (MBR). Formula: A line segment formed by 2 points (x_1, y_1) and (x_2, y_2) will have an MBR with the bottom-left corner at $(\min(x_1, x_2), \min(y_1, y_2))$ and the top-right corner at $(\max(x_1, x_2), \max(y_1, y_2))$.
- **R-Tree Query:** Insert all n MBRs into the R-Tree. Using the R-Tree's Window Query algorithm, we quickly identify the MBRs that intersect with the query rectangle.
- **Result:** The set of line segments corresponding to the retrieved MBRs is called the Candidate set.
- **Complexity:** Using the R-Tree reduces the search cost to only $\mathcal{O}(\log n + k)$, where k is the number of line segments in the candidate set. Since $k \ll n$, this operation eliminates most unnecessary data, speeding up the query thousands of times.

Step 2: Refinement Step

- **Question:** Why do we still need to iterate and re-check after finding the candidate set?
- **Explanation:** This step is mandatory to eliminate False hits. This situation occurs when the MBR of a line segment intersects with the query window, but the actual line segment inside that MBR does not touch the query window at all.
- **Operation:** Scan through the candidate set (with a very small size k) and use a geometric algorithm to check the exact intersection between the line segment and the query window. The segments that truly intersect will be selected to be drawn on the screen.

5.2 Exercise 2: Region Query on a Set of Rectangles

Given a set S consisting of rectangles with edges parallel to the coordinate axes. Given a query rectangle r, return all rectangles in S that are covered by r.

Solution:

1. Data Structure

- **Internal Node:** Stores Minimum Bounding Rectangles (MBRs) and pointers to child nodes.
- **Leaf Node:** Directly stores the coordinates of the data rectangles belonging to set S (each rectangle is defined by 4 boundaries: left, right, top, bottom).

2. Insertion & Splitting

- **Choose Subtree:** When adding a new rectangle to the tree, traverse from the root downwards. At each internal node, choose the child branch whose MBR requires the least "perimeter enlargement" to fully enclose the new rectangle.
- **Overflow Handling:** When a leaf node exceeds its maximum capacity, it is mandatory to use an Internal-node Split Algorithm to split it in half. Reason: The data at the leaf node is now a rectangle (with area and 4 edges) rather than a point (1D coordinate). The algorithm will sort the list of rectangles sequentially by the left, right, top, and bottom boundaries along the X and Y axes. The system will try cutting at valid positions (ensuring each group has at least $0.4B$ elements) to find a split that minimizes the total perimeter of the 2 newly generated MBRs.

3. Querying

Start the recursive search function from the root node with the query rectangle r:

- **Step 1 - Filtering at Internal Node:** Iterate through the child branches. Check whether the MBR of that child branch intersects or is covered by r.
  - If yes: Recursively traverse down that branch.
  - If no: Prune and completely skip that branch to save time.
- **Step 2 - Checking at Leaf Node:** Upon reaching a leaf node, iterate through the stored data rectangles. Check the condition if the data rectangle lies completely inside r:
  - Condition: `data.left >= r.left` AND `data.right <= r.right` AND `data.bottom >= r.bottom` AND `data.top <= r.top`.
  - If all are satisfied, add that rectangle to the result list.

4. Complexity

- The average search cost is reduced to $\mathcal{O}(\log n + k)$, where n is the total number of rectangles in the tree and k is the number of rectangles within the region r (the returned results).

5.3 Exercise 3: K-Nearest Neighbors (KNN)

Problem Statement: Given a set S of N data points on a 2D plane indexed by an R-Tree. Given a query point P and a positive integer K. Find K points in S that have the shortest Euclidean distance to P.

Detailed Solution:

Instead of a Window Query like the previous exercises, the KNN problem uses the Best-First Search algorithm combined with a Priority Queue (Min-Heap) to traverse the R-Tree.

1. Core Concept: Minimum Distance (MinDist)

- For the algorithm to know which branch to descend first, we must calculate the distance from the query point P to the MBRs of the R-Tree.
- MinDist(P, MBR) is the shortest distance from point P to any point lying on the boundary or inside that MBR.

- If P lies completely inside the MBR, MinDist = 0. Thanks to MinDist, we can sort and prioritize opening the branches most likely to contain the points closest to P.

2. Algorithm Execution Steps:

- **Step 1 (Initialization):** Create a Priority Queue (Min-Heap) to store the nodes of the R-Tree, sorted in ascending order by MinDist value. Initially, push the root node of the R-Tree into the queue. Create an empty list to store the K results.
- **Step 2 (Traversing Min-Heap):** Repeat the following process until the queue is empty or K result points have been collected: Extract the element with the smallest MinDist from the top of the queue.
- **Step 3 (Processing the extracted element):**
  - **Case A (It is an actual data point):** Since it was extracted from the Min-Heap, it is certainly the closest point to P currently. Add this point to the result list. If the list has enough K elements, the algorithm terminates.
  - **Case B (It is an internal node / MBR):** This node contains child branches. Calculate the MinDist from P to all child MBRs (or child points) of that node. Push all these children back into the Min-Heap. The algorithm automatically resorts so the closest element always bubbles to the top.

3. Complexity Evaluation:

- This algorithm is highly optimal because it exploits the "pruning" property of the R-Tree. MBRs located too far from P will have a very large MinDist, sink to the bottom of the Min-Heap, and almost never be extracted for traversal (avoiding scanning all N points).
- The average complexity of this query is $\mathcal{O}(\log N + K)$.

5.4 Exercise 4: Point Enclosure Query (Application in Mouse Click Detection on UI)

Problem Statement: In User Interface (UI) programming, components like panels, windows, or buttons are often represented by Bounding Boxes and can overlap. Given a set S consisting of N rectangles representing UI components. Given Q queries, each corresponding to a mouse click at coordinates $P(x, y)$. Find all rectangles in S that enclose point P.

Detailed Solution:

Approach 1: Brute-force

- **Method:** For each point $P(x, y)$, iterate through all N rectangles. Point P is considered inside the rectangle if its coordinates satisfy the condition: $x_{min} \leq x \leq x_{max}$ and $y_{min} \leq y \leq y_{max}$.
- **Complexity:** $\mathcal{O}(N)$ per query. If the system has a large number of UI elements or complex polygons, this method will cause latency during user interaction.

Approach 2: Applying R-Tree

This problem is the inverse of Exercise 2. Instead of using a region to find objects inside it, we use a point to find the regions covering it.

- **Data Structure:** Build an R-Tree where the leaf nodes store the coordinates of the N UI rectangles.
- **Query Operation:** A recursive search function starting from the root node:
  - **At the Internal Node:** For each child branch, check whether point $P(x, y)$ lies inside the MBR of that branch.
    - If P is not inside the MBR, P definitely cannot be inside any element belonging to that branch $\rightarrow$ Completely prune this branch.
    - If P is inside the MBR, recursively traverse down the child branch.
  - **At the Leaf Node:** Scan through the stored data rectangles. Check whether point P truly lies inside that rectangle (checking 4 boundaries). If yes, add this rectangle to the result list.

- **Complexity:** By pruning irrelevant UI areas, the R-Tree narrows down the search scope extremely fast. The average time to process each mouse click drops to $\mathcal{O}(\log N + k)$, where k is the number of UI components overlapping at the exact coordinate P. The interaction response process occurs in approximately $\mathcal{O}(1)$ in practice.

Bibliography

[1] Antonin Guttman. *R-Trees: A Dynamic Index Structure for Spatial Searching*. Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data, pp. 47–57, 1984[cite: 833]. *Source: http://db.deis.unibo.it/courses/SI-LS/papers/Gut84.pdf*

[2] Yufei Tao. *The R-Tree*. Lecture Notes for INFS4205/7205, University of Queensland[cite: 2, 5]. *Source: https://www.cse.cuhk.edu.hk/ taoyf/course/infs4205/lec/rtree.pdf*

[3] Yufei Tao. *Spatial Databases - Lecture 13: R-Tree*. Course Notes, The Chinese University of Hong Kong. *Source: https://www.cse.cuhk.edu.hk/ taoyf/course/wst501/notes/lec13.pdf*

[4] Joyeeta Arulraj. *Database Systems Implementation - Lecture 17: R-Trees*. Georgia Institute of Technology, Fall 2024. *Source: https://faculty.cc.gatech.edu/ jarulraj/courses/4420-f24/slides/17-rtree.pdf*